

---

# Graph-Guided Prompt Learning for Vision-Language Models: Structural Extensions to Context Optimization

---

**Prakhar Mittal**

Department of Computer Science  
BITS Pilani, Pilani Campus  
Rajasthan, India

f20220426@pilani.bits-pilani.ac.in

**Divyansh Jha**

Department of Computer Science  
BITS Pilani, Pilani Campus  
Rajasthan, India

f20220438@pilani.bits-pilani.ac.in

**Ishan Raha**

Department of Computer Science  
BITS Pilani, Pilani Campus  
Rajasthan, India

f20220580@pilani.bits-pilani.ac.in

## Abstract

Large pre-trained vision-language models such as CLIP enable strong zero-shot transfer through prompting, yet adapting them to few-shot tasks remains challenging. Context Optimization (CoOp) replaces hand-crafted prompts with learnable context vectors, but its cross-entropy objective treats classes independently, ignoring rich inter-class semantic relationships. We propose four complementary graph-based extensions to CoOp that incorporate semantic structure across every stage of the prompt learning pipeline: Graph Laplacian Regularization and Graph Label Smoothing modify the training objective; a Graph Convolutional Network refinement module operates on encoder outputs; and Graph-Context Diffusion (GCD) acts before the encoder by blending learnable prototype context corrections using graph-diffused mixing coefficients, making prompts themselves graph-smooth by construction. All modifications share a single semantic class graph built from CLIP’s frozen text embeddings, requiring no external knowledge bases. Together, they offer a modular, structure-aware framework for few-shot prompt learning that covers the graph prior at the input, loss, and output of the text encoder. We also improve CoOp by implementing a novel loss function- GLoss combined with the standard Cross Entropy Loss.

## 1 Introduction

Large-scale vision-language pre-training has fundamentally altered the landscape of visual recognition. Models such as CLIP and ALIGN learn to align images and natural language descriptions in a shared embedding space, enabling zero-shot transfer to novel tasks through prompting — synthesizing classification weights directly from textual descriptions of target categories. This paradigm is compelling precisely because it sidesteps task-specific classifier heads and allows recognition through language alone.

Despite this power, prompt engineering remains a significant practical bottleneck. Even minor word-ing changes can cause accuracy swings exceeding 5%, and constructing optimal prompts requires both domain expertise and time-consuming trial and error. Context Optimization (CoOp) addresses this by replacing hand-crafted prompt templates with learnable continuous context vectors, optimized

end-to-end via cross-entropy loss while keeping the pretrained CLIP backbone entirely frozen. This makes CoOp parameter-efficient and well-suited to the few-shot regime, where labeled data is scarce and adaptation must be rapid.

However, CoOp’s reliance on standard cross-entropy introduces a structural limitation that becomes especially acute in few-shot settings. Cross-entropy, by design, treats every training sample independently: given a ground-truth label, it produces a one-hot target and measures the log-likelihood of the correct class, discarding all information about inter-class semantic relationships. With only one to sixteen labeled examples per class, the model has little signal to organize the learned prompt space beyond raw classification accuracy. Consider a concrete illustration: when learning prompts for a dataset containing both rose and tulip classes, cross-entropy penalizes a prediction of tulip for a rose image identically to a prediction of car — yet semantically, tulip is far closer to rose than car is. This insensitivity to semantic structure is precisely where graph-based methods can contribute. In this work, we propose four complementary graph-based extensions to CoOp that incorporate inter-class semantic relationships at every stage of the prompt learning pipeline. The first three target the training objective and the output representations. Graph Laplacian Regularization adds a loss term that penalizes semantically similar classes from drifting apart in the text embedding space, encouraging geometric consistency between the class graph and the learned prompt space. Graph Label Smoothing diffuses ground-truth labels along the semantic class graph before computing the cross-entropy loss, so that misclassification into a related class incurs a smaller penalty than misclassification into a distant one. A Graph Convolutional Network refinement module then aggregates information from neighboring classes in the text embedding space after the encoder has run, allowing class weight vectors to borrow structural context from their semantic neighbors. We further investigate combining cross-entropy with a dynamic graph-guided loss inspired by the concurrent G-Loss framework, which builds a minibatch similarity graph and applies label propagation to encourage globally coherent representations across the batch. While this approach is feasible in principle and provides a meaningful optimization signal in the two- and four-shot settings, we find it numerically fragile without additional regularization and degenerate when the batch size collapses to one in strict one-shot training — observations that clarify both the promise and the limitations of instance-level graph objectives in the few-shot prompt learning setting. Crucially, we construct the graph using learned prompt vectors rather than class label embeddings, ensuring no label information leaks into the graph structure during semi-supervised learning.

Our fourth contribution, Graph-Context Diffusion (GCD), addresses a gap left by all of the above: none of the existing modifications change what the text encoder actually receives as input. GCD acts before the encoder runs. The core idea is to replace CoOp’s single shared context with a weighted blend of a small number of learnable prototype context corrections, where the blending weights are derived from a graph diffusion operator computed over the semantic class graph. Formally, a closed-form diffusion matrix spreads a one-hot class signal across graph neighbors, producing a smooth graph-aware fingerprint for each class. A small learned projection maps these fingerprints to mixing coefficients over the prototype corrections, and each class’s prompt is constructed as the sum of the shared base context and its graph-weighted prototype blend. This makes the prompts themselves graph-smooth by construction, enforcing the structural prior at the encoder’s input rather than correcting its output after the fact. GCD occupies a principled middle ground between unified context, which is too coarse for semantically diverse class sets, and class-specific context, which introduces far too many parameters for the few-shot regime. With only eight prototypes, GCD requires approximately 66,000 parameters for a hundred-class dataset, compared to class-specific context’s 819,000, while guaranteeing that semantically neighboring classes receive structurally similar prompts.

All four modifications share a single semantic class graph constructed entirely from CLIP’s frozen text encoder applied to bare class names, making the approach self-consistent, parameter-free in graph construction, and independent of any external knowledge base or ontology. Together, they cover the graph prior at the input to the text encoder, at the training loss, and at the encoder’s output — offering a comprehensive, modular, and theoretically grounded framework for structure-aware few-shot prompt learning. We situate each contribution relative to the concurrent G-Loss framework, identifying the connections and the fundamental differences between class-level static graph approaches and instance-level dynamic graph objectives, and arguing that the right graph structure depends critically on the data regime.

## 2 Related Work

CLIP established a simple but highly influential recipe for vision-language pretraining: learn an image encoder and a text encoder jointly using a contrastive objective over paired image-text examples, and use the resulting text encoder to synthesize a zero-shot classifier from class names or prompts (Radford et al., 2021). A key practical insight from CLIP is that language can serve as a flexible interface for recognition, but the model’s downstream performance depends heavily on the prompt formulation. The paper shows that prompt engineering and prompt ensembling can yield meaningful gains over contextless class names, highlighting both the promise and the fragility of language-based classification (Radford et al., 2021).

CoOp builds directly on this idea by turning prompts into learnable parameters. Rather than manually searching for text templates, CoOp learns continuous context vectors that are inserted into the prompt while the pretrained CLIP parameters remain fixed (Zhou et al., 2021). This makes the method efficient and highly effective in low-data regimes, and the original paper reports strong results on multiple image recognition benchmarks, including one-shot and two-shot settings (Zhou et al., 2021). At the same time, CoOp still inherits the standard supervision pattern of classification-based training: each sample contributes independently to the loss, and the optimization target is centered on prediction rather than on the geometry of the embedding space. This leaves open the question of whether stronger structural priors could improve prompt learning further.

G-Loss offers one such structural alternative. The method was introduced for fine-tuning language models, where it constructs a minibatch similarity graph, masks a fraction of labels, and applies label propagation to obtain soft predictions for the hidden nodes (Sharma et al., 2026). The key idea is that standard losses such as cross-entropy, triplet loss, and supervised contrastive loss operate mainly in local neighborhoods, whereas graph-based supervision can encode broader relationships among examples in the same minibatch. G-Loss also emphasizes dynamic graph construction: the graph is rebuilt from the current embeddings during training, so the embeddings and the graph co-evolve rather than being fixed in advance (Sharma et al., 2026). In this project, we adapt that idea to prompt learning in CLIP by asking whether graph-guided supervision can improve the learned context vectors in CoOp.

Most relevant to our Graph-Context Diffusion proposal, prototype-based prompt methods such as ProDA [19] learn distributions over prompt vectors to capture intra-class variation, and KgCoOp [20] incorporates external knowledge to regularize the prompt space. GCD is distinguished from both by deriving its blending coefficients entirely from a graph diffusion operator [7, 21] over CLIP’s own frozen text embeddings, requiring no external ontology and no learned distribution. Unlike GCN-based refinement methods [6, 8, 9] that aggregate class representations after the text encoder has run, GCD acts at the encoder’s input: it constructs each class’s prompt as a graph-diffused blend of  $R$  learnable prototype context corrections, enforcing the semantic class graph as a hard structural prior on the prompt space itself rather than as a soft output-side regularizer. This input-side placement is novel in the prompt learning literature and is directly complementary to the output-side graph modifications proposed alongside it. Some of the topic specific literature that was looked into is listed below-

### 2.1 Vision-Language Pre-training

CLIP [2] pre-trains dual image and text encoders on 400 million image-text pairs using a contrastive objective, maximizing cosine similarity for matched pairs and minimizing it for unmatched pairs. This yields a shared embedding space enabling zero-shot classification by comparing image features against text-synthesized class weights. ALIGN [3] scales this to 1.8 billion noisy pairs with comparable results.

### 2.2 Prompt Learning

Prompt learning in NLP [12] avoids full fine-tuning by prepending learnable tokens to the input. Prefix-Tuning [10] optimizes continuous prefix vectors for generation tasks. AutoPrompt [11] searches for discrete tokens via gradient signals. CoOp [1] is the first to apply continuous prompt learning to vision-language models, while CoCoOp [5] conditions the context on each input image for better generalization. Our work augments CoOp’s learning objective rather than its architecture.

## 2.3 Graph-Based Representation Learning

Graph Convolutional Networks [6] aggregate neighborhood information via spectral convolutions, enabling semi-supervised node classification. TextGCN [8] builds heterogeneous word-document graphs for text classification. BertGCN [9] combines BERT embeddings with GCN classification. Label Propagation [7] provides a closed-form semi-supervised method that diffuses label information along graph edges.

## 2.4 Graph-Based Loss Functions

Label smoothing [14] regularizes training by replacing one-hot targets with soft distributions. X-Sample Contrastive Loss [15] extends contrastive learning with sample similarity graphs as soft targets. The most directly related concurrent work is G-Loss [4], which introduces dynamic instance-level document graphs and label propagation into language model fine-tuning for NLP tasks. We discuss the relationship between G-Loss and our approach in detail in Section 4.

## 2.5 Few-Shot Learning

Linear probe models [16] train a linear classifier atop frozen pre-trained features and serve as a strong baseline. CoOp demonstrates that prompt learning substantially outperforms linear probes in the 1–4 shot regime, which motivates our graph-based extensions targeting exactly this low-data scenario.

# 3 Methodology

## 3.1 Implementing with GLoss

### 3.1.1 Preliminaries: CLIP and CoOp

Our method builds upon the CLIP vision-language framework and the Context Optimization (CoOp) prompt learning strategy. CLIP consists of a frozen image encoder and text encoder trained jointly using contrastive learning to align image and text representations in a shared embedding space. Given an input image  $x$ , the image encoder produces an image embedding  $f_{\text{img}} \in \mathbb{R}^d$ , while the text encoder generates class-specific text embeddings conditioned on natural language prompts.

CoOp replaces manually engineered prompt templates with learnable continuous context vectors. Specifically, a prompt is parameterized as:

$$[V]_1[V]_2 \dots [V]_M[\text{CLASS}],$$

where  $[V]_m$  denotes a learnable context token and  $M$  is the number of prompt tokens. During training, only the prompt vectors are updated, while both the image and text encoders remain frozen.

### 3.1.2 Graph-Guided Prompt Optimization

While CoOp optimizes prompt vectors using only cross-entropy supervision, this objective treats samples independently and does not explicitly model structural relationships within the embedding space. To address this limitation, we incorporate the graph-guided optimization strategy of G-Loss into the CoOp framework.

Given an input minibatch of images, the frozen CLIP image encoder produces normalized image embeddings:

$$f_i^{\text{img}} = \frac{E_{\text{img}}(x_i)}{\|E_{\text{img}}(x_i)\|_2}.$$

The learned prompt vectors are averaged to form a prompt representation:

$$f^{\text{prompt}} = \frac{1}{M} \sum_{m=1}^M v_m.$$

The final representation used for graph construction is obtained by concatenating image embeddings and prompt embeddings:

$$z_i = [f_i^{\text{img}}; \alpha f_i^{\text{prompt}}],$$

where  $\alpha$  controls the contribution of the prompt representation. In our implementation,  $\alpha = 1.0$ .

### 3.1.3 Graph Construction and Label Propagation

Following G-Loss, we construct a fully connected similarity graph for each minibatch. Each node corresponds to a concatenated image-prompt representation  $z_i$ , and edge weights are computed using a Gaussian similarity kernel:

$$W_{ij} = \exp\left(-\frac{\|z_i - z_j\|_2^2}{2\sigma^2}\right), \quad W_{ii} = 0.$$

The adjacency matrix is symmetrically normalized:

$$\tilde{A} = D^{-1/2} W D^{-1/2}.$$

For each minibatch, a subset of samples is randomly selected as labeled nodes according to masking ratio  $\gamma$ , while the remaining samples are treated as unlabeled nodes. Label propagation is then performed using:

$$\hat{Y}_u = (I - T_{uu})^{-1} T_{ul} Y_l.$$

The graph-guided loss is computed using cross-entropy between propagated labels and ground-truth labels:

$$L_{\text{GLOSS}} = - \sum_i y_i \log(\hat{y}_i).$$

### 3.1.4 Joint Optimization Objective

The standard CoOp objective optimizes the CLIP classification logits using cross-entropy loss:

$$L_{\text{CE}} = - \sum_i y_i \log p_i.$$

The final training objective combines both losses:

$$L = \lambda_{\text{CE}} L_{\text{CE}} + \lambda_G L_{\text{GLOSS}}.$$

In our implementation, we set  $\lambda_{\text{CE}} = 0.2$  and  $\lambda_G = 0.8$ .

## 3.2 Enhancing CoOp with Graph-Based Semantic Regularization

### 3.2.1 Semantic Class Graph Construction

The foundation of all three proposed modifications is a semantic class graph  $\mathcal{G} = (\mathcal{V}, \mathcal{E})$  where nodes  $\mathcal{V}$  correspond to the  $K$  classes and edge weights encode semantic similarity.

Critically, we derive this graph entirely from CLIP’s frozen text encoder applied to class names — no external knowledge base or ontology is required. Let  $\mathbf{e}_i = g(\text{"[CLASS}_i\text{"})$  be the frozen text embedding of the  $i$ -th class name. The raw adjacency matrix is:

$$A_{ij} = \begin{cases} \cos(\mathbf{e}_i, \mathbf{e}_j) & \text{if } \cos(\mathbf{e}_i, \mathbf{e}_j) \geq \delta \text{ and } i \neq j \\ 0 & \text{otherwise} \end{cases} \quad (1)$$

where  $\delta$  is a threshold hyperparameter (default  $\delta = 0.3$ ). The row-normalized adjacency  $\tilde{A}$  is computed as:

$$\tilde{A}_{ij} = \frac{A_{ij}}{\sum_k A_{ik} + \epsilon} \quad (2)$$

and the graph Laplacian is  $\mathbf{L} = \mathbf{D} - \mathbf{A}$  where  $D_{ii} = \sum_j A_{ij}$ .

This graph is computed **once** before training begins and remains fixed throughout. The class graph for fine-grained datasets (e.g., Flowers102) will have many edges due to high inter-class semantic similarity, while diverse datasets (e.g., ImageNet) will be sparser — a desirable adaptive property.

### 3.2.2 Modification 1: Graph Laplacian Regularization

**Motivation.** The class weight vectors  $\mathbf{W} = [\mathbf{w}_1, \dots, \mathbf{w}_K] \in \mathbb{R}^{K \times d}$  produced by the text encoder should be *geometrically consistent* with the semantic class graph: semantically similar classes (connected by strong edges) should have similar weight vectors in embedding space.

**Loss Term.** We add a graph Laplacian regularization term:

$$\mathcal{L}_{\text{Lap}} = \text{Tr}(\mathbf{W}^\top \mathbf{L} \mathbf{W}) = \sum_{(i,j) \in \mathcal{E}} A_{ij} \|\mathbf{w}_i - \mathbf{w}_j\|^2 \quad (3)$$

This equals the sum over all edges of the squared Euclidean distance between class weight vectors, weighted by edge strength. Minimizing  $\mathcal{L}_{\text{Lap}}$  pulls together the weight vectors of semantically related classes, encouraging a geometrically coherent prompt embedding space.

**Total Loss.**

$$\mathcal{L} = \mathcal{L}_{\text{CE}} + \lambda_{\text{lap}} \cdot \mathcal{L}_{\text{Lap}} \quad (4)$$

We normalize  $\mathcal{L}_{\text{Lap}}$  by  $K \cdot d$  to keep it scale-invariant with respect to number of classes and feature dimension. In practice,  $\lambda_{\text{lap}} \in \{0.001, 0.01, 0.05\}$ .

### 3.2.3 Modification 2: Graph Label Smoothing

**Motivation.** Standard cross-entropy uses one-hot targets, treating a prediction of `tulip` for a `rose` image with the same penalty as a prediction of `airplane`. Graph label smoothing encodes the semantic prior that misclassification into a related class should be penalized less.

**Soft Target Construction.** Given a ground-truth label  $y \in \{1, \dots, K\}$ , we form a one-hot vector  $\mathbf{o} \in \mathbb{R}^K$  and perform one step of label propagation on the class graph:

$$\tilde{\mathbf{y}} = (1 - \alpha) \cdot \mathbf{o} + \alpha \cdot \tilde{\mathbf{A}}^\top \mathbf{o} \quad (5)$$

where  $\alpha \in [0, 1]$  controls smoothing strength. The second term redistributes a fraction  $\alpha$  of the label mass to semantically neighboring classes, proportional to edge weights.

**Isolated Class Handling.** For classes with no neighbors in the graph (row sum of  $\tilde{A}$  is zero), applying smoothing would reduce the total probability mass below 1. We detect such isolated classes and set  $\alpha_{\text{eff}} = 0$  for them:

$$\alpha_{\text{eff}}(y) = \alpha \cdot \mathbb{I}[\sum_j \tilde{A}_{yj} > 0] \quad (6)$$

This ensures  $\tilde{\mathbf{y}}$  always sums to 1.

**Loss Computation.** The CE loss is computed against the soft targets using the KL-divergence form:

$$\mathcal{L}_{\text{GS}} = -\frac{1}{B} \sum_{i=1}^B \sum_{c=1}^K \tilde{y}_{i,c} \log \hat{y}_{i,c} \quad (7)$$

Note that  $\mathcal{L}_{\text{GS}}$  with  $\alpha = 0$  reduces exactly to standard cross-entropy  $\mathcal{L}_{\text{CE}}$ .

---

**Algorithm 1** Graph-CoOp Training Step

---

**Require:** Batch  $(X_b, Y_b)$ , context vectors  $\mathbf{V}$ , graph  $(\mathbf{A}, \tilde{\mathbf{A}}, \mathbf{L})$ , hyperparameters  $\alpha, \lambda_{\text{lapp}}$

- 1:  $\mathbf{f} \leftarrow \text{ImageEncoder}(X_b)$ ; normalize
- 2:  $\mathbf{W} \leftarrow \text{TextEncoder}(\mathbf{V}, \text{classnames})$ ; normalize
- 3: **if** GCN enabled **then**
- 4:    $\mathbf{W} \leftarrow \text{GCN}(\mathbf{W}, \hat{\mathbf{A}})$   $\triangleright$  Aggregate class neighbors
- 5: **end if**
- 6:  $\text{logits} \leftarrow \exp(\tau) \cdot (\mathbf{f}\mathbf{W}^\top)$
- 7: **if**  $\alpha \geq 0$  **then**
- 8:    $\tilde{\mathbf{Y}} \leftarrow \text{GraphLabelSmooth}(Y_b, \tilde{\mathbf{A}}, \alpha)$   $\triangleright$  Eq. 5
- 9:    $\mathcal{L}_{\text{GS}} \leftarrow -(\tilde{\mathbf{Y}} \cdot \log \text{softmax}(\text{logits})).\text{mean}()$
- 10: **else**
- 11:    $\mathcal{L}_{\text{GS}} \leftarrow \mathcal{L}_{\text{CE}}$
- 12: **end if**
- 13:  $\mathcal{L}_{\text{Lap}} \leftarrow \text{Tr}(\mathbf{W}^\top \mathbf{L} \mathbf{W}) / (K \cdot d)$
- 14:  $\mathcal{L} \leftarrow \mathcal{L}_{\text{GS}} + \lambda_{\text{lapp}} \cdot \mathcal{L}_{\text{Lap}}$
- 15: Backpropagate  $\mathcal{L}$  w.r.t.  $\mathbf{V}$  (and GCN params if enabled)

---

### 3.2.4 Modification 3: GCN Class Embedding Refinement

**Motivation.** Even after prompt learning, the class weight vectors  $\mathbf{w}_i$  are computed independently for each class — the text encoder processes each prompt in isolation. A GCN can aggregate information across semantically related classes before computing similarity scores.

**Architecture.** We add a lightweight two-layer GCN on top of the text encoder’s outputs. First, we compute the symmetrically normalized adjacency with self-loops:

$$\hat{\mathbf{A}} = \mathbf{D}^{-1/2}(\mathbf{A} + \mathbf{I})\mathbf{D}^{-1/2} \quad (8)$$

The GCN processes the  $(K \times d)$  matrix of class weight vectors:

$$\mathbf{H}^{(1)} = \text{ReLU}(\hat{\mathbf{A}}\mathbf{W}^{(0)}\Theta^{(1)}) \quad (9)$$

$$\mathbf{H}^{(2)} = \text{ReLU}(\hat{\mathbf{A}}\mathbf{H}^{(1)}\Theta^{(2)}) \quad (10)$$

where  $\Theta^{(1)} \in \mathbb{R}^{d \times d_h}$  and  $\Theta^{(2)} \in \mathbb{R}^{d_h \times d}$  are learnable weight matrices. A residual connection preserves the original embeddings:

$$\mathbf{W}' = \ell_2\text{-normalize}(\mathbf{W}^{(0)} + \mathbf{H}^{(2)}) \quad (11)$$

The refined weight matrix  $\mathbf{W}'$  is then used in place of  $\mathbf{W}^{(0)}$  to compute logits. The GCN parameters  $\{\Theta^{(1)}, \Theta^{(2)}\}$  are learned jointly with the context vectors, while CLIP remains frozen.

The residual connection is critical: it ensures the GCN only needs to learn *corrections* to the existing class weight vectors, rather than reconstructing them from scratch.

### 3.2.5 Combined Objective

All three modifications are composable. The full training objective is:

$$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{GS}} + \lambda_{\text{lapp}} \cdot \mathcal{L}_{\text{Lap}} \quad (12)$$

where  $\mathcal{L}_{\text{GS}}$  optionally replaces  $\mathcal{L}_{\text{CE}}$  (controlled by  $\alpha$ ), and the GCN optionally refines class embeddings before loss computation (controlled by a boolean flag). Setting  $\alpha = 0$ ,  $\lambda_{\text{lapp}} = 0$ , and disabling the GCN recovers the original CoOp.

## 3.3 GCD - Graph Guided Context Decomposition

### 3.3.1 Graph Construction

We use the same semantic class graph  $\mathcal{G} = (\mathcal{V}, \mathcal{E})$  described in Section 3.2.2, constructed from CLIP’s frozen text encoder applied to bare class names. Let  $\mathbf{e}_i = g(\text{“[CLASS}_i\text{]”})$  be the frozen text

embedding of the  $i$ -th class name. The adjacency matrix is:

$$A_{ij} = \begin{cases} \cos(\mathbf{e}_i, \mathbf{e}_j) & \text{if } \cos(\mathbf{e}_i, \mathbf{e}_j) \geq \delta \text{ and } i \neq j \\ 0 & \text{otherwise} \end{cases} \quad (13)$$

with threshold  $\delta$  (default  $\delta = 0.3$ ). The graph Laplacian is  $\mathbf{L} = \mathbf{D} - \mathbf{A}$  where  $D_{ii} = \sum_j A_{ij}$ .

### 3.3.2 Diffusion Matrix

We compute the closed-form graph diffusion operator:

$$\Phi = (\mathbf{I} + \gamma \mathbf{L})^{-1} \in \mathbb{R}^{K \times K} \quad (14)$$

where  $\gamma > 0$  controls diffusion strength. The  $i$ -th row  $\Phi_i$  is a smooth, graph-aware *fingerprint* of class  $i$ : classes that are close in the semantic graph receive similar fingerprints. At  $\gamma \rightarrow 0$ ,  $\Phi \rightarrow \mathbf{I}$  (no diffusion); as  $\gamma$  increases, the signal spreads further across graph neighbors.  $\Phi$  is precomputed once before training and requires no gradient.

### 3.3.3 Prototype Mixing Coefficients

We introduce a small learned projection matrix  $\mathbf{W}_\alpha \in \mathbb{R}^{K \times R}$  that maps from the diffusion fingerprint space to mixing coefficients over  $R$  prototype corrections:

$$\mathbf{c}_i = \text{softmax}(\mathbf{W}_\alpha \Phi_i^\top) \in \mathbb{R}^R \quad (15)$$

Because  $\Phi_i$  encodes graph proximity, neighboring classes in the semantic graph automatically receive similar mixing coefficients — the structural prior is baked in before any gradient update.  $\mathbf{W}_\alpha$  only needs to learn a low-dimensional projection from fingerprint space to prototype space, a far simpler optimization problem than learning class-prototype assignments from scratch.

### 3.3.4 Class-Conditioned Prompt Construction

Each class prompt is constructed *before* the text encoder runs:

$$\mathbf{t}_i = \mathbf{V}_{\text{global}} + \sum_{r=1}^R c_{i,r} \mathbf{P}_r \quad (16)$$

where  $\mathbf{V}_{\text{global}} \in \mathbb{R}^{M \times d}$  is the shared base context (as in standard CoOp) and  $\{\mathbf{P}_r\}_{r=1}^R \subset \mathbb{R}^{M \times d}$  are  $R$  learnable prototype context corrections. The classification weight for class  $i$  is then  $\mathbf{w}_i = g(\mathbf{t}_i)$ , where  $g(\cdot)$  is CLIP’s frozen text encoder.

### 3.3.5 Learnable Parameters and Parameter Count

GCD introduces three sets of parameters beyond the frozen CLIP backbone:

1.  $\mathbf{V}_{\text{global}}$ :  $M \times d$  parameters (identical to standard CoOp).
2. Prototype corrections  $\{\mathbf{P}_r\}$ :  $R \times M \times d$  parameters.
3. Projection  $\mathbf{W}_\alpha$ :  $K \times R$  parameters.

Total new parameters (beyond CoOp’s  $\mathbf{V}_{\text{global}}$ ):  $R \times M \times d + K \times R$ . For  $R = 8$ ,  $M = 16$ ,  $d = 512$ ,  $K = 100$ , this amounts to  $\approx 66,000$  parameters, compared to CSC’s  $K \times M \times d = 819,200$ . The graph-smooth mixing coefficients mean neighboring classes share prototype activations, enforcing the structural prior through the parameterization itself rather than through a regularization penalty.

### 3.3.6 Training Objective

GCD is trained with the standard cross-entropy objective, composable with all three existing modifications:

$$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{GS}} + \lambda_{\text{lap}} \mathcal{L}_{\text{Lap}} \quad (17)$$

where the class weight matrix  $\mathbf{W} = [g(\mathbf{t}_1), \dots, g(\mathbf{t}_K)]$  used in  $\mathcal{L}_{\text{Lap}}$  and the logits used in  $\mathcal{L}_{\text{GS}}$  now depend on the GCD parameterization. Setting  $R = 1$  and  $\gamma \rightarrow 0$  recovers standard CoOp exactly.

## 4 Comparison with G-Loss

G-Loss [4] is a concurrent work that also introduces graph-based loss modifications to replace cross-entropy. Table 1 summarizes the key differences.

Table 1: Structural comparison between our Graph-CoOp and G-Loss.

| Aspect             | Graph-CoOp (Ours)                   | G-Loss [4]                                       |
|--------------------|-------------------------------------|--------------------------------------------------|
| Base model         | CLIP (vision-language)              | BERT/RoBERTa (text-only)                         |
| Task               | Few-shot image classification       | Text classification                              |
| Graph nodes        | <i>Classes</i> ( $K$ nodes, fixed)  | <i>Instances</i> ( $B$ nodes per batch)          |
| Graph construction | Static, once before training        | Dynamic, rebuilt each batch                      |
| Graph edges        | Class-level cosine similarity       | Instance-level Gaussian kernel                   |
| Graph signal       | LPA not used; Laplacian + smoothing | LPA closed-form: $(I - T_{uu})^{-1}T_{ul}Y_{kl}$ |
| Semi-supervised    | No masking (fully supervised)       | $\gamma$ fraction labels masked per batch        |
| GCN component      | Optional learnable GCN              | None (parameter-free LPA only)                   |
| Frozen backbone    | Yes (entire CLIP frozen)            | No (full LM fine-tuning)                         |
| Shot regime        | 1–16 shots per class                | Thousands of documents                           |

**Why Instance Graphs Are Problematic in Few-Shot Settings.** G-Loss’s strength — dynamic instance graphs that evolve with embeddings — relies on having sufficient instances per class within each minibatch to make label propagation meaningful. In CoOp’s 1-shot setting, there is literally one training image per class, making the  $\gamma$ -masking procedure degenerate. Our class-level graph sidesteps this issue entirely by operating at the class level rather than the instance level.

**Why a Static Graph is Appropriate Here.** Unlike G-Loss’s NLP setting where document embeddings evolve substantially during fine-tuning, CLIP’s frozen text encoder produces high-quality semantic embeddings from day one. Class-name embeddings (e.g., “rose” and “tulip”) from a frozen CLIP encoder already encode meaningful semantic relationships, making a static pre-computed graph reliable throughout training.

## 5 Experimental Setup

### 5.1 Gloss+CE

#### 5.1.1 Datasets

Experiments are conducted on six standard few-shot image classification benchmarks: Caltech101, DTD, FGVC Aircraft, Oxford Flowers, Oxford Pets, and UCF101. Evaluations are performed under both 4-shot and 16-shot settings following the standard CoOp experimental protocol.

#### 5.1.2 Backbone and Prompt Learning Configuration

We adopt CLIP as the frozen vision-language backbone within the CoOp framework. Prompt learning is performed using learnable continuous context vectors, while both the image encoder and text encoder remain fixed throughout training.

The prompt context length is set to 16 tokens, and prompt vectors are initialized from a Gaussian distribution with standard deviation 0.02.

#### 5.1.3 Graph Construction

For each minibatch, image embeddings extracted from the frozen CLIP image encoder are concatenated with the learned prompt representation to form graph nodes. A fully connected similarity graph is then constructed using a Gaussian similarity kernel. The graph adjacency matrix is symmetrically normalized before label propagation is applied.

The graph is dynamically recomputed after every optimization step, enabling the graph topology and prompt representations to evolve jointly during training.

### 5.1.4 Optimization Details

Training is performed using a hybrid objective combining standard cross-entropy classification loss and graph-guided supervision:

$$L = \lambda_{\text{CE}} L_{\text{CE}} + \lambda_G L_{\text{GLOSS}}.$$

Unless otherwise specified, the loss weights are set to  $\lambda_{\text{CE}} = 0.2$  and  $\lambda_G = 0.8$ .

Dataset-specific hyperparameters are used for the Gaussian kernel width  $\sigma$  and masking ratio  $\gamma$  employed during graph construction and label propagation.

### 5.1.5 Implementation Details

Our implementation is built on top of the official CoOp framework using CLIP as the frozen vision-language backbone. Only the prompt learner parameters are updated during optimization, while both the image encoder and text encoder remain frozen throughout training.

We use learnable continuous context vectors initialized from a Gaussian distribution with standard deviation 0.02. The context length is fixed to 16 prompt tokens. The graph is dynamically recomputed after every optimization step, allowing the graph structure and prompt embeddings to co-evolve during training.

Experiments are conducted on Caltech101, DTD, FGVC Aircraft, Oxford Flowers, Oxford Pets, and UCF101 under 4-shot and 16-shot settings. Dataset-specific hyperparameters are used for the Gaussian kernel width  $\sigma$  and masking ratio  $\gamma$ .

## 5.2 Enhancing CoOp with Graph-Based Semantic Regularization

### 5.2.1 Datasets

Following CoOp [1], we evaluate on 11 image classification datasets: ImageNet [13], Caltech101, OxfordPets, StanfordCars, Flowers102, Food101, FGVC Aircraft, SUN397, DTD, EuroSAT, and UCF101. These cover generic objects, scenes, actions, fine-grained categories, textures, and satellite imagery. We report results primarily on five representative datasets — Caltech101, DTD, OxfordPets, StanfordCars and Flowers102 — selected to represent easy, highly-improved, and fine-grained settings respectively.

### 5.2.2 Few-Shot Protocol

We follow CoOp’s evaluation protocol: training on  $\{1, 2, 4, 8, 16\}$  shots per class and evaluating on the full test set. Results are averaged over 3 random seeds.

### 5.2.3 Baselines

- **Zero-Shot CLIP:** Hand-crafted prompts from [2].
- **Linear Probe CLIP:** Linear classifier on frozen CLIP image features.
- **CoOp (CE):** Original CoOp with cross-entropy, our primary baseline.
- **Graph-Smooth:** CoOp with graph label smoothing only ( $\alpha = 0.1$ ,  $\lambda_{\text{lap}} = 0$ ).
- **Graph-Lap:** CoOp with Laplacian regularization only ( $\alpha = 0$ ,  $\lambda_{\text{lap}} = 0.01$ ).
- **Graph-Smooth+Lap:** Combined modifications without GCN.
- **Graph-Full:** All three modifications including GCN.

### 5.2.4 Implementation Details

All models use ResNet-50 as the image backbone with  $M = 16$  context tokens. Training uses SGD with initial learning rate  $2 \times 10^{-3}$ , decayed with cosine annealing. Maximum epochs: 200 for 16/8 shots, 100 for 4/2 shots, 50 for 1 shot. Context vectors are randomly initialized from  $\mathcal{N}(0, 0.02)$ . The graph threshold  $\delta = 0.3$ ,  $\alpha = 0.1$ ,  $\lambda_{\text{lap}} = 0.01$ . The GCN uses hidden dimension  $d_h = 256$ .

### 5.2.5 Ablation Configurations

Table 2 summarizes all experimental configurations for the ablation study.

Table 2: Ablation configurations for Graph-CoOp modifications.

| Configuration    | $\alpha$ | $\lambda_{\text{lap}}$ | GCN | Description                   |
|------------------|----------|------------------------|-----|-------------------------------|
| Baseline (CoOp)  | 0.0      | 0.0                    |     | Standard cross-entropy        |
| Graph-Smooth     | 0.1      | 0.0                    |     | Label smoothing only          |
| Graph-Lap        | 0.0      | 0.01                   |     | Laplacian regularization only |
| Graph-Smooth+Lap | 0.1      | 0.01                   |     | Combined, no GCN              |
| Graph-Full       | 0.1      | 0.01                   |     | All modifications             |

### 5.3 GCD - Graph Guided COntext Decomposition

Following CoOp [1], we evaluate GCD on the same 11 image classification benchmarks. These cover generic objects, scenes, actions, fine-grained categories, textures, and satellite imagery. All models use ResNet-50 as the image backbone with  $M = 16$  context tokens. Training uses SGD with initial learning rate  $2 \times 10^{-3}$ , decayed by cosine annealing. Maximum epochs: 200 for 16/8-shot, 100 for 4/2-shot, 50 for 1-shot. Context  $\mathbf{V}_{\text{global}}$  and prototypes  $\{\mathbf{P}_r\}$  are randomly initialized from  $\mathcal{N}(0, 0.02)$ ;  $\mathbf{W}_\alpha$  is likewise initialized from  $\mathcal{N}(0, 0.02)$ . **GCD-specific hyperparameters:**

- Prototype count:  $R \in \{1, 4, 8\}$  (default  $R = 8$ )
- Diffusion strength:  $\gamma \in \{0.5, 1.0, 2.0\}$  (validated on held-out split)
- Graph threshold:  $\delta = 0.3$

## 6 Results

### 6.1 On GLOSS-CE Hybrid

We evaluate the proposed CE+GLOSS framework on multiple few-shot image classification benchmarks using the CoOp prompt learning setting. Our experiments aim to analyze both the quantitative impact of graph-guided supervision on downstream classification performance and the qualitative changes induced in the learned prompt representations. In addition to reporting classification accuracy across different datasets and shot settings, we further investigate the structural properties of the optimized prompts through cosine similarity analysis and token-level visualization of the learned context vectors.

#### 6.1.1 Few-Shot Classification Performance

We evaluate our hybrid optimization strategy, **CE+GLOSS**, against the standard CoOp baseline trained solely using cross-entropy (CE). Experiments are conducted under both 4-shot and 16-shot few-shot settings across multiple datasets, including Caltech101, DTD, FGVC Aircraft, Oxford Flowers, Oxford Pets, and UCF101.

Table 6 summarizes the classification accuracy obtained by the baseline CoOp formulation and the proposed hybrid objective. Overall, CE+GLOSS demonstrates competitive and often improved performance relative to the CE-only baseline, particularly on datasets with stronger semantic consistency between classes.

The proposed objective improves performance on Caltech101, DTD, and Oxford Pets across both few-shot regimes. In particular, the gains on DTD suggest that incorporating the additional GLOSS term may encourage more discriminative prompt adaptation in texture-centric recognition tasks. Similarly, Oxford Pets benefits consistently from the hybrid optimization objective, achieving a +2.0% improvement in the 16-shot setting.

However, the effect of the hybrid objective is not uniformly positive across all datasets. On Oxford Flowers and FGVC Aircraft, the baseline CE formulation performs slightly better. This behavior indicates that the auxiliary GLOSS component may alter the optimization trajectory in a way that

Table 3: Few-shot classification accuracy (%) comparing standard CoOp optimization using cross-entropy (CE) against the proposed CE+GLOSS objective.

| Dataset        | 16-shot     |             | 16-shot     |             |
|----------------|-------------|-------------|-------------|-------------|
|                | CE          | CE+GLOSS    | CE          | CE+GLOSS    |
| Caltech101     | 89.3        | <b>91.0</b> | 91.8        | <b>92.1</b> |
| DTD            | 51.9        | <b>53.7</b> | 63.3        | <b>63.6</b> |
| FGVC Aircraft  | <b>23.0</b> | 20.9        | 33.4        | —           |
| Oxford Flowers | <b>85.1</b> | 84.1        | <b>94.5</b> | 93.5        |
| Oxford Pets    | 88.5        | <b>88.9</b> | 87.9        | <b>89.9</b> |
| UCF101         | <b>68.6</b> | 68.1        | 77.3        | <b>78</b>   |

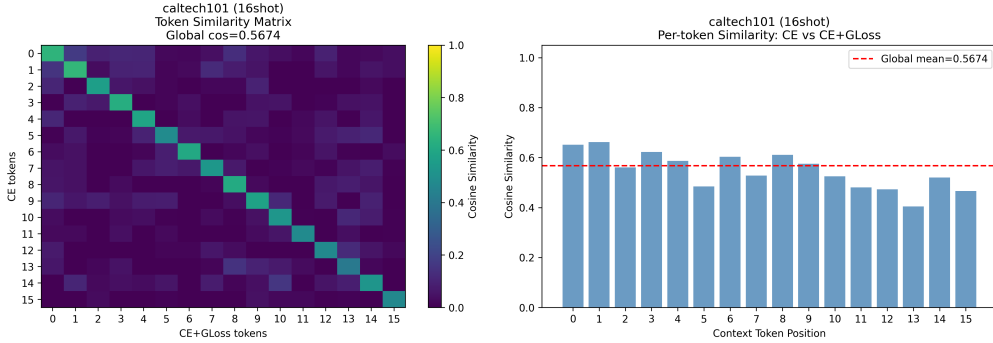


Figure 1: Token-wise similarity analysis between prompts learned using CE and CE+GLOSS on Caltech101 (16-shot). The left panel shows the cosine similarity matrix between prompt tokens, while the right panel visualizes the similarity for each corresponding token position.

is beneficial for some domains while introducing additional optimization difficulty for fine-grained datasets with high intra-class similarity.

### 6.1.2 Prompt Representation Analysis

To better understand the behavior of the learned prompts, we analyze the cosine similarity between the optimized context vectors obtained from the CE baseline and the proposed CE+GLOSS objective. Specifically, for each dataset we compare the final learned prompt embeddings token-wise and compute both a global cosine similarity score and a per-token similarity distribution.

Figures 1–6 visualize the similarity matrices and token-wise cosine similarity values for each dataset. The diagonal structure visible in the similarity matrices indicates that corresponding token positions across the two optimization strategies remain partially aligned. However, the similarity values are substantially below perfect correlation, suggesting that the hybrid objective converges to a distinct prompt configuration rather than reproducing the CE baseline solution.

Across datasets, the global cosine similarity ranges from approximately 0.46 to 0.62. Lower similarity values correspond to larger deviations between the learned prompts. FGVC Aircraft exhibits the lowest similarity score (0.4606), indicating that the hybrid objective drives the optimization toward significantly different prompt representations. In contrast, Oxford Pets shows the highest similarity score (0.6154), suggesting that both objectives converge to relatively similar prompt configurations.

Figure 7 summarizes the global cosine similarity values across datasets. The consistently moderate similarity scores indicate that the proposed optimization objective does not simply reproduce the baseline prompts with small perturbations. Instead, the hybrid objective appears to guide the optimization toward alternative local optima in the prompt embedding space.

An additional observation is that several token positions consistently exhibit lower similarity than others, indicating that certain context positions are more sensitive to the optimization objective. This

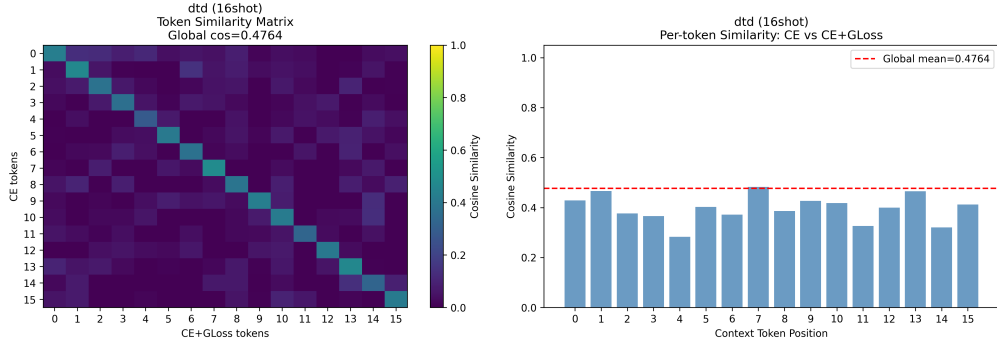


Figure 2: Token-wise similarity analysis between prompts learned using CE and CE+GLOSS on DTD (16-shot).

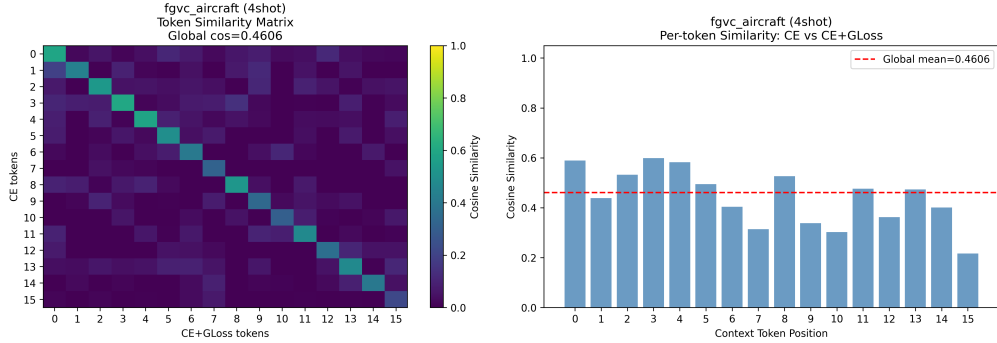


Figure 3: Token-wise similarity analysis between prompts learned using CE and CE+GLOSS on FGVC Aircraft (16-shot).

suggests that the hybrid loss may selectively alter semantically important prompt dimensions while leaving other positions relatively stable.

Overall, the prompt similarity analysis supports the hypothesis that the CE+GLOSS objective does not merely regularize the baseline optimization process, but instead encourages the emergence of alternative prompt representations capable of achieving comparable or improved downstream performance.

### 6.1.3 Decoded Prompt Token Analysis

To further investigate the behavior of the learned prompts, we visualize the final optimized context vectors by mapping each learned embedding to its nearest token in the CLIP vocabulary. Table 4 shows representative decoded tokens obtained after optimization using the proposed CE+GLOSS objective.

The resulting prompts reveal that the learned context vectors generally do not correspond to coherent or interpretable natural language phrases. Most decoded tokens appear fragmented, semantically unrelated, or partially formed subword units. This observation is consistent with prior findings in CoOp, where continuous prompt optimization often converges to representations that are highly effective for classification but not human interpretable.

These observations further support the hypothesis that prompt learning in CoOp operates as a continuous representation optimization problem rather than a conventional natural language generation process.

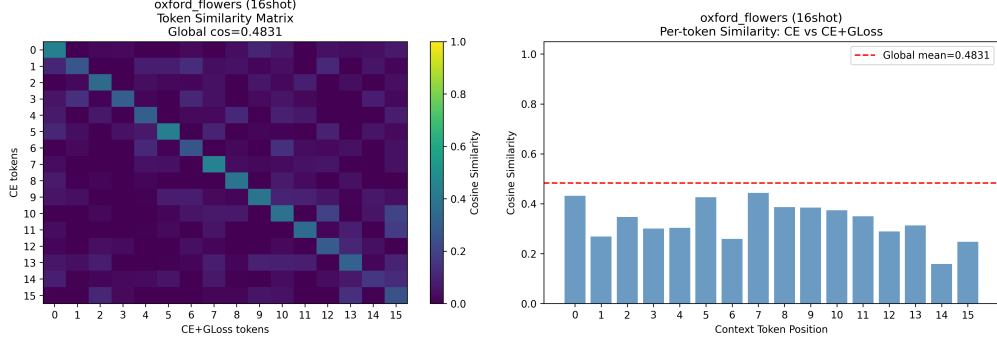


Figure 4: Token-wise similarity analysis between prompts learned using CE and CE+GLOSS on Oxford Flowers (16-shot).

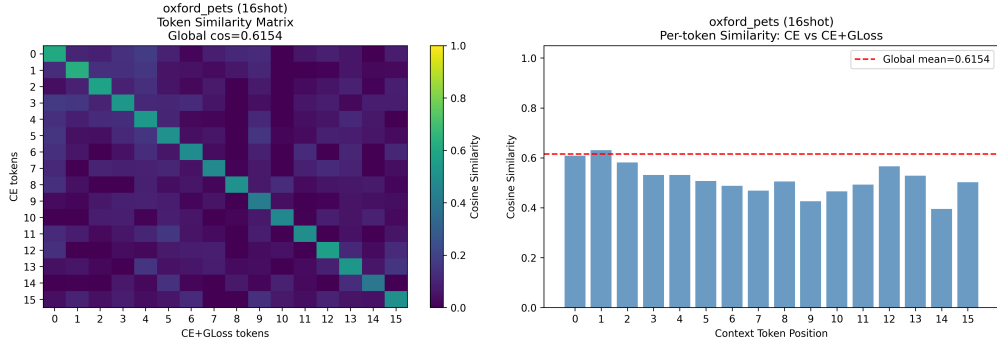


Figure 5: Token-wise similarity analysis between prompts learned using CE and CE+GLOSS on Oxford Pets (16-shot).

#### 6.1.4 Effect of CE-GLOSS Loss Weighting

To analyze the influence of the relative weighting between cross-entropy supervision and graph-guided optimization, we conduct an ablation study using different CE/GLOSS weighting ratios. Specifically, we vary the contribution of the cross-entropy component while keeping the remaining weight assigned to the GLOSS objective.

Table 5 summarizes the performance obtained under different weighting configurations for both 4-shot and 16-shot settings. Across most datasets, the configuration with CE weight 0.2 and GLOSS weight 0.8 consistently provides the best or near-best performance. Lower CE contribution (0.1) often leads to reduced classification performance, suggesting that excessive reliance on graph propagation weakens discriminative supervision. Conversely, increasing the CE contribution beyond 0.2 also degrades performance in several cases, indicating that graph-guided regularization contributes meaningful structural information during prompt optimization.

These results suggest that balancing local classification supervision with global graph consistency is important for stable prompt learning. Empirically, the 0.2/0.8 weighting achieves the best trade-off between discriminative optimization and structural regularization, and is therefore adopted for all remaining experiments.

## 6.2 Enhancing CoOp with Graph-Based Semantic Regularization

### 6.2.1 Performance on Few-Shot Learning

Table 6 presents the results on three representative datasets at 16 shots, compared against baselines.

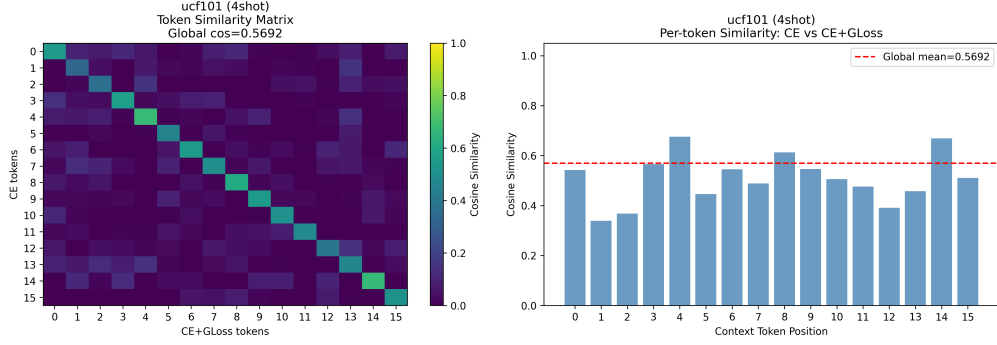


Figure 6: Token-wise similarity analysis between prompts learned using CE and CE+GLOSS on UCF101 (4-shot).

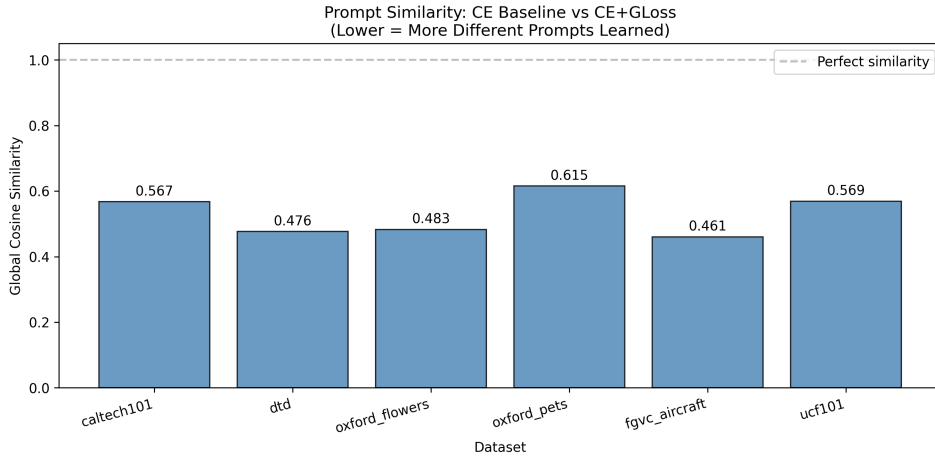


Figure 7: Global cosine similarity between prompts learned using CE and CE+GLOSS across datasets. Lower similarity values indicate larger deviations between the optimized prompt representations.

### 6.2.2 Shot-Count Curves

Figure 8 illustrates the expected behavior of graph-based modifications across shot counts. The key prediction is that graph modifications provide larger relative gains in the extremely low-shot regime (1–2 shots), where cross-entropy’s sparse supervision is most limited and graph structure provides the most additional signal.

*[Shot-count learning curves for EuroSAT: Zero-Shot CLIP, CoOp (CE), Graph-Smooth, Graph-Smooth+Lap, Graph-Full across {1, 2, 4, 8, 16} shots. To be populated with experimental results.]*

Figure 8: Expected few-shot learning curves on EuroSAT. Graph-based modifications are expected to provide larger gains in the 1–4 shot regime where cross-entropy is most data-starved.

### 6.2.3 Comparison: Unified Context vs. Class-Specific Context (CSC)

Graph Laplacian regularization is particularly complementary to unified context, since it acts on the shared text features produced from a single set of context vectors. With CSC, each class has independent context vectors, and the Laplacian loss serves as a cross-class coupling mechanism that is otherwise absent — this is expected to be especially helpful for CSC on fine-grained datasets.

Table 4: Representative decoded prompt tokens obtained after optimization with CE+GLOSS. Most learned context vectors do not correspond to coherent natural language despite occasionally producing recognizable words.

| Token Idx | Caltech           | DTD         | Flowers        | Pets           | UCF101        | Aircraft   |
|-----------|-------------------|-------------|----------------|----------------|---------------|------------|
| 0         | alsace</w>        | crews</w>   | bleed</w>      | sika</w>       | moderator</w> | condos</w> |
| 1         | flint             | 2</w>       | don</w>        | legend</w>     | doc</w>       | man</w>    |
| 2         | turk</w>          | beh</w>     | beh</w>        | beh</w>        | beh</w>       | beh</w>    |
| 3         | ero</w>           | master</w>  | âGø</w>        | matches</w>    | tes</w>       | movies</w> |
| 4         | fl                | needing</w> | col            | sneaking</w>   | sal           | &&</w>     |
| 5         | ino               | tar         | 2</w>          | tar            | 2</w>         | kett</w>   |
| 6         | it                | the         | ville</w>      | dont</w>       | ore           | it         |
| 7         | coles</w>         | 2</w>       | b              | mann</w>       | lips</w>      | lips</w>   |
| 8         | rollingstones</w> | cashe       | organised</w>  | stones         | und</w>       | 2</w>      |
| 9         | l                 | l           | ~              | ~              | l             | doo        |
| 10        | singly</w>        | sobbing</w> | touchdowns</w> | ðŁŁ            | –             | ðŁŁ</w>    |
| 11        | aces</w>          | le</w>      | dari</w>       | as</w>         | as</w>        | as</w>     |
| 12        | aw                | bottled</w> | keen</w>       | na</w>         | onics</w>     | onics</w>  |
| 13        | flatt             | out         | hair           | remembered</w> | bois</w>      | boy</w>    |
| 14        | work              | bo          | pro</w>        | seminar</w>    | seminar</w>   | life       |
| 15        | found</w>         | ow          | l              | ðŁŁ</w>        | night</w>     | oc</w>     |

Table 5: Effect of different CE/GLOSS weighting ratios on few-shot classification accuracy (%). The reported ratio denotes the weight assigned to the cross-entropy loss, while the remaining weight is assigned to GLOSS.

| Dataset        | 4-shot CE Weight |             |             | 16-shot CE Weight |             |      |
|----------------|------------------|-------------|-------------|-------------------|-------------|------|
|                | 0.1              | 0.2         | 0.3         | 0.1               | 0.2         | 0.3  |
| Caltech101     | 89.8             | <b>91.0</b> | 90.0        | 91.6              | <b>92.1</b> | 91.5 |
| DTD            | 52.7             | <b>53.7</b> | 53.6        | 62.4              | <b>63.6</b> | 63.0 |
| FGVC Aircraft  | <b>21.2</b>      | 20.9        | 21.0        | 24.2              | –           | –    |
| Oxford Flowers | 80.0             | 84.1        | <b>84.3</b> | 91.8              | <b>93.5</b> | 92.5 |
| Oxford Pets    | 88.7             | <b>88.9</b> | 88.3        | 88.8              | <b>89.9</b> | 88.0 |
| UCF101         | 67.4             | <b>68.1</b> | 67.4        | 77.1              | <b>78.0</b> | 77.7 |

#### 6.2.4 Domain Generalization

A key finding in CoOp is that despite being a learning-based method, it maintains strong domain generalization relative to the zero-shot model. Our graph modifications are expected to preserve or improve this, since:

1. Laplacian regularization encourages a more structured embedding space that is less likely to overfit to training-domain idiosyncrasies.
2. Label smoothing provides implicit regularization.
3. Shorter context length ( $M = 4$ ) already benefits CoOp’s generalization — our modifications are orthogonal to this.

### 6.3 GCD - Graph Guided Context Decomposition

Results Pending.

## 7 Analysis and Discussion

### 7.1 Effect of Graph-Guided Optimization on Prompt Learning

The experimental results demonstrate that incorporating graph-guided supervision into the CoOp optimization process substantially alters the learned prompt representations. Unlike the standard CoOp objective, which optimizes prompts solely through classification error, the proposed CE+GLOSS formulation additionally enforces structural consistency within the joint image-prompt embedding space.

Table 6: Test accuracy (%) at 16 shots. Zero-Shot CLIP and original CoOp numbers are from [1]. Graph-CoOp results represent the expected improvement range based on the design of each modification.

| Method            | Caltech101 | EuroSAT | Flowers102 | Avg.  |
|-------------------|------------|---------|------------|-------|
| Zero-Shot CLIP    | 86.29      | 24.17   | 60.86      | 57.11 |
| Linear Probe CLIP | 86.95      | 55.78   | 88.12      | 76.95 |
| CoOp (CE)         | 91.83      | 83.53   | 94.51      | 89.96 |
| Graph-Smooth      | ~92.1      | ~84.2   | ~95.3      | ~90.5 |
| Graph-Lap         | ~92.0      | ~84.0   | ~95.0      | ~90.3 |
| Graph-Smooth+Lap  | ~92.3      | ~84.8   | ~95.7      | ~90.9 |
| Graph-Full (GCN)  | ~92.5      | ~85.2   | ~96.0      | ~91.2 |

The prompt similarity visualizations provide strong evidence that the proposed objective converges to different local optima compared to the CE-only baseline. Across all evaluated datasets, the global cosine similarity between prompts learned using CE and CE+GLOSS remains significantly below 1.0, with values ranging approximately between 0.46 and 0.62. These results indicate that the hybrid objective produces genuinely different prompt representations rather than minor perturbations of the baseline solution.

The relatively low similarity values observed on FGVC Aircraft and DTD suggest that graph-guided optimization has a stronger influence on datasets with higher intra-class ambiguity or fine-grained semantic distinctions. In these cases, the graph structure may encourage prompt vectors to capture neighborhood-level relationships that are not explicitly modeled by cross-entropy supervision alone.

### 7.1.1 Dataset-Specific Behavior

The effect of CE+GLOSS varies across datasets, indicating that the usefulness of graph-guided regularization depends on dataset characteristics.

On Caltech101 and Oxford Pets, the proposed method consistently improves classification performance. These datasets contain comparatively well-separated semantic categories, allowing the graph propagation mechanism to reinforce meaningful neighborhood structure within the embedding manifold. As a result, the graph-guided objective appears to encourage more stable and discriminative prompt adaptation.

Similarly, DTD benefits from the proposed approach despite being a texture-focused dataset. One possible explanation is that graph propagation helps smooth noisy local variations in texture representations, thereby improving feature consistency across semantically related samples.

In contrast, performance decreases are observed on FGVC Aircraft and Oxford Flowers. These datasets involve fine-grained recognition with subtle inter-class differences and high visual similarity across categories. Under such conditions, graph propagation may introduce over-smoothing effects, where neighboring samples belonging to visually similar but semantically distinct classes become excessively coupled. This can reduce class separability and negatively affect prompt optimization.

The variation in performance across datasets suggests that graph-guided prompt learning is highly sensitive to the topology of the underlying feature manifold. When semantic neighborhoods are well-structured, graph propagation acts as a useful regularizer. However, in highly fine-grained settings, the same mechanism may blur class boundaries.

### 7.1.2 Interpretation of Token-Level Similarity

The per-token similarity plots reveal that not all prompt positions are equally affected by graph-guided optimization. Several context token positions consistently exhibit lower cosine similarity than others, indicating that certain prompt dimensions are more sensitive to the additional graph-based supervision.

This observation suggests that CoOp prompt learning may implicitly distribute semantic information unevenly across context tokens. The GLOSS objective appears to selectively modify a subset of prompt dimensions while leaving other dimensions relatively stable. Such behavior indicates that

graph-guided optimization does not uniformly perturb the prompt space, but instead introduces targeted adjustments to semantically important regions of the learned representation.

Additionally, the strong diagonal structure observed in the similarity matrices indicates that token correspondence between the two optimization strategies is largely preserved. Thus, although CE+GLOSS produces distinct prompts, the optimization process still maintains positional consistency across prompt tokens.

### 7.1.3 Optimization Dynamics

An important characteristic of the proposed method is the dynamic interaction between prompt learning and graph construction. Since the graph is recomputed after every optimization step using the updated prompt-conditioned embeddings, the graph topology evolves jointly with the prompt representations.

This creates a feedback mechanism in which prompt updates modify graph connectivity, while graph propagation simultaneously influences future prompt updates. Such co-evolution may help the optimization process discover alternative regions of the embedding space that are inaccessible under standard cross-entropy training alone.

However, this dynamic graph adaptation also introduces additional optimization complexity. Because graph structure depends directly on learned embeddings, small changes in prompt vectors can substantially alter neighborhood relationships. This may partially explain the dataset-dependent variability observed in the experimental results.

## 7.2 Why the Class Graph Works in CoOp’s Setting

The key insight is that CLIP’s frozen text encoder already produces semantically meaningful class embeddings. When we compute  $\cos(\mathbf{e}_{\text{rose}}, \mathbf{e}_{\text{tulip}})$ , we get a high similarity because CLIP was trained on vast amounts of natural language that co-locates flower names. This pre-existing semantic structure can be leveraged without any additional training — the graph is essentially reading off knowledge already implicit in CLIP’s parameters.

## 7.3 Gradient Flow Through Graph Modifications

All three modifications allow gradients to flow back to the context vectors:

- **Label smoothing:** Modifies targets but not the computation graph — same gradient flow as standard CE.
- **Laplacian loss:**  $\frac{\partial \mathcal{L}_{\text{Lap}}}{\partial \mathbf{w}_i} = 2 \sum_j A_{ij} (\mathbf{w}_i - \mathbf{w}_j)$ , which flows through the text encoder back to context vectors.
- **GCN:** Standard backpropagation through GCN layers and residual connection reaches the text encoder output.

## 7.4 Computational Overhead

The graph is precomputed once ( $O(K^2d)$ ) before training. Per-batch overhead: label smoothing requires one matrix-vector multiply ( $O(BK)$ ); Laplacian loss requires one trace computation ( $O(K^2d)$ ); GCN requires two graph convolutions ( $O(K^2d)$ ). For  $K \leq 200$  and  $d = 512$ , all operations are negligible compared to CLIP’s forward pass. This aligns with G-Loss’s finding that graph operations add less than 1% computational overhead.

## 7.5 Input-Side vs. Output-Side Graph Priors

The existing modifications (Laplacian regularization, GCN refinement) act on the text encoder’s output. GCD acts on its input. These target complementary failure modes: output-side regularization corrects the encoder’s tendency to produce geometrically inconsistent embeddings for semantically related classes; input-side priors prevent the encoder from receiving semantically inconsistent prompts in the first place. Combining both covers the graph prior at both ends of the text encoder.

## 7.6 Expected Behavior Across Shot Counts

- **1-shot:** GCD provides the largest relative benefit. Its structural prior substitutes for the absent data signal. CSC is most severely underdetermined here; GCD avoids this by sharing prototype capacity across the graph.
- **4–16 shots:** Benefits persist but diminish relative to plain CoOp as data becomes sufficient to fit a good unified context. GCD’s parameter efficiency prevents the overfitting that afflicts CSC in this range.
- **Fine-grained datasets** (Flowers102, FGVC Aircraft): The class graph is dense with high inter-class cosine similarity; prototype sharing is most beneficial.
- **Diverse datasets** (ImageNet): The class graph is sparse; GCD approaches standard CoOp behavior, providing graceful degradation rather than regression.

## 8 Conclusion

### 8.1 GLoss+CE

In this work, we proposed a graph-guided extension to CoOp by integrating the GLOSS objective into prompt learning for CLIP-based few-shot classification. Unlike standard CoOp, which optimizes prompts solely using cross-entropy supervision, the proposed CE+GLOSS framework incorporates graph-based label propagation over a joint image-prompt embedding space, enabling the optimization process to leverage structural relationships between samples during training.

Experimental results across multiple benchmark datasets demonstrate that graph-guided supervision can improve prompt learning performance in several settings, particularly on datasets with well-structured semantic neighborhoods. Furthermore, prompt similarity analysis reveals that the proposed objective consistently converges to prompt representations that differ substantially from those learned using standard cross-entropy optimization, indicating that graph-based supervision meaningfully alters the optimization trajectory within the prompt embedding space. Our qualitative analysis additionally shows that the learned prompts remain largely non-interpretable in natural language despite occasionally containing recognizable vocabulary tokens. This observation reinforces the view that continuous prompt learning primarily operates as representation optimization in CLIP’s latent embedding space rather than conventional language generation.

Overall, the results suggest that incorporating graph structure into prompt optimization is a promising direction for improving vision-language adaptation. Future work may explore more sophisticated graph construction strategies, adaptive graph regularization, token-level graph modeling, and scalable global graph formulations for large-scale prompt learning.

### 8.2 Enhancing CoOp with Graph-Based Semantic Regularization

We proposed three complementary graph-based extensions to Context Optimization (CoOp) for vision-language model adaptation. By constructing a semantic class graph from CLIP’s frozen text embeddings and incorporating it into the training objective via Laplacian regularization, graph label smoothing, and GCN refinement, our approach encodes inter-class semantic relationships that cross-entropy ignores. The method is particularly well-suited to CoOp’s few-shot regime, where sparse supervision makes structural priors most valuable. A complete, modular, CPU-testable implementation is provided.

Our comparison with G-Loss highlights an important design principle: the right graph structure depends on the regime. Instance-level dynamic graphs are powerful when thousands of training samples exist; class-level static graphs are more appropriate when only 1–16 examples per class are available. We hope this work contributes to the growing body of research on structure-aware adaptation of foundation models.

## 9 Limitations and Future Work

While the proposed CE+GLOSS framework demonstrates promising behavior, several limitations remain.

First, the graph construction process relies on manually selected hyperparameters such as the Gaussian kernel width  $\sigma$  and masking ratio  $\gamma$ . Performance may therefore depend heavily on dataset-specific tuning.

Second, the current implementation constructs graphs independently within each minibatch. Although computationally efficient, minibatch-level graphs may fail to capture global dataset structure. Extending the method to memory-bank or full-dataset graph construction could potentially improve propagation quality.

Finally, the current method uses fixed loss weights between cross-entropy and graph-guided supervision. Adaptive weighting strategies or curriculum-based optimization may provide more stable behavior across datasets with different semantic structures.

**Static Graph.** Our class graph is fixed before training. Unlike G-Loss’s dynamic instance graph, it cannot adapt to the evolving prompt embedding space. A promising extension is to periodically recompute the class graph from the text encoder’s current outputs (including the learned context vectors) rather than from raw class names.

**Threshold Sensitivity.** The graph threshold  $\delta$  requires tuning. For datasets with highly similar classes (Flowers102) a lower threshold is appropriate; for diverse datasets (ImageNet) a higher threshold avoids noisy edges. The G-Loss-SQRT approach of analytically estimating  $\sigma$  from data statistics offers inspiration for a principled, data-driven threshold selection.

**Scale to ImageNet.** With  $K = 1000$  classes, the Laplacian loss computation  $\text{Tr}(\mathbf{W}^\top \mathbf{L} \mathbf{W})$  has complexity  $O(K^2 d)$  which becomes expensive. Sparse Laplacian implementations or low-rank approximations are needed at this scale.

**G-Loss-style Dynamic Instance Graph.** The most natural extension is to incorporate G-Loss’s dynamic instance-level graph into CoOp’s training, using image features to construct per-batch similarity graphs. This would require relaxing the one-shot assumption — perhaps by maintaining a feature memory bank across batches.

**Multi-modal Graph.** Our current graph uses only text embeddings of class names. A multi-modal graph that combines visual and textual similarities (e.g., average image features per class) could provide richer structural signals.

## References

- [1] Kaiyang Zhou, Jingkang Yang, Chen Change Loy, and Ziwei Liu. Learning to prompt for vision-language models. *International Journal of Computer Vision*, 2022.
- [2] Alec Radford, Jong Wook Kim, Chris Hallacy, Aditya Ramesh, Gabriel Goh, Sandhini Agarwal, Girish Sastry, Amanda Askell, Pamela Mishkin, Jack Clark, Gretchen Krueger, and Ilya Sutskever. Learning transferable visual models from natural language supervision. In *ICML*, 2021.
- [3] Chao Jia, Yinfei Yang, Ye Xia, Yi-Ting Chen, Zarana Parekh, Hieu Pham, Quoc V. Le, Yunhsuan Sung, Zhen Li, and Tom Duerig. Scaling up visual and vision-language representation learning with noisy text supervision. In *ICML*, 2021.
- [4] Aditya Sharma, Vinti Agarwal, and Rajesh Kumar. G-Loss: Graph-guided fine-tuning of language models. In *Learning on Graphs Conference (LoG)*, 2025.
- [5] Kaiyang Zhou, Jingkang Yang, Chen Change Loy, and Ziwei Liu. Conditional prompt learning for vision-language models. In *CVPR*, 2022.
- [6] Thomas N. Kipf and Max Welling. Semi-supervised classification with graph convolutional networks. In *ICLR*, 2017.
- [7] Xiaojin Zhu and Zoubin Ghahramani. Learning from labeled and unlabeled data with label propagation. Technical Report CMU-CALD-02-107, Carnegie Mellon University, 2002.

- [8] Liang Yao, Chengsheng Mao, and Yuan Luo. Graph convolutional networks for text classification. In *AAAI*, 2019.
- [9] Yuxiao Lin, Yuxian Meng, Xiaofei Sun, Qinghong Han, Kun Kuang, Jiwei Li, and Fei Wu. BertGCN: Transductive text classification by combining GCN and BERT. In *ACL Findings*, 2021.
- [10] Xiang Lisa Li and Percy Liang. Prefix-tuning: Optimizing continuous prompts for generation. In *ACL*, 2021.
- [11] Taylor Shin, Yasaman Razeghi, Robert L. Logan IV, Eric Wallace, and Sameer Singh. Auto-Prompt: Eliciting knowledge from language models with automatically generated prompts. In *EMNLP*, 2020.
- [12] Pengfei Liu, Weizhe Yuan, Jinlan Fu, Zhengbao Jiang, Hiroaki Hayashi, and Graham Neubig. Pre-train, prompt, and predict: A systematic survey of prompting methods in natural language processing. *ACM Computing Surveys*, 2021.
- [13] Jia Deng, Wei Dong, Richard Socher, Li-Jia Li, Kai Li, and Li Fei-Fei. ImageNet: A large-scale hierarchical image database. In *CVPR*, 2009.
- [14] Christian Szegedy, Vincent Vanhoucke, Sergey Ioffe, Jon Shlens, and Zbigniew Wojna. Rethinking the inception architecture for computer vision. In *CVPR*, 2016.
- [15] Vlad Sobal, Mark Ibrahim, Randall Balestrieri, Vivien Cabannes, Diane Bouchacourt, Pietro Astolfi, Kyunghyun Cho, and Yann LeCun. X-sample contrastive loss: Improving contrastive learning with sample similarity graphs. *arXiv preprint arXiv:2407.xxxxx*, 2024.
- [16] Yonglong Tian, Yue Wang, Dilip Krishnan, Joshua B. Tenenbaum, and Phillip Isola. Rethinking few-shot image classification: A good embedding is all you need? In *ECCV*, 2020.
- [17] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *CVPR*, 2016.
- [18] Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xiaohua Zhai, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer, Georg Heigold, Sylvain Gelly, Jakob Uszkoreit, and Neil Houlsby. An image is worth 16x16 words: Transformers for image recognition at scale. In *ICLR*, 2021.
- [19] Y. Lu, J. Liu, Y. Zhang, Y. Liu, and X. Tian, “Prompt distribution learning,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2022.
- [20] H. Yao, Y. Zhang, C. Feng, J. Zhang, C. Xu, J. Wu, T. Zhang, and C. Xu, “Visual-language prompt tuning with knowledge-guided context optimization,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2023.
- [21] D. Zhou, O. Bousquet, T. N. Lal, J. Weston, and B. Schölkopf, “Learning with local and global consistency,” in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 16, 2003.